IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 4 August 2025, accepted 5 September 2025, date of publication 9 September 2025, date of current version 15 September 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3607879

RESEARCH ARTICLE

G-SQL: A Schema-Aware and Rule-Guided Approach for Robust Natural Language to SQL Translation

HASSAN S. SHALAAN[1], TAYSEER HASSAN A. SOLIMAN[1], (Member, IEEE), AND AMR M. ABDELAZIZ[2]

[1]Faculty of Computers and Information, Assiut University, Asyut 71515, Egypt
[2]Faculty of Computers and Artificial Intelligence, Beni-Suef University, Beni Suef 62521, Egypt

Corresponding author: Hassan S. Shalaan (hassanshalaan@aun.edu.eg)

ABSTRACT This paper introduces G-SQL, a schema-aware and rule-guided framework for translating Natural Language Queries (NLQ) into SQL, designed to support users with limited technical expertise. G-SQL bridges the limitations of traditional rule-based and deep learning approaches by combining JSON-based schema serialization with Natural Language Processing (NLP) tools, such as GloVe and Stanford CoreNLP. The system encodes database schemas—including tables, columns, keys, and relationships—into a structured JSON format, enabling efficient and interpretable mapping between query tokens and schema elements. SQL statements are generated using a guided template-driven strategy. Evaluated across three benchmark datasets (IMDB, Yelp, MAS), For easy queries, it achieved 100% execution accuracy on all three datasets. On medium queries, G-SQL reached 95% on IMDB, 85% on Yelp, and 90% on MAS. For hard queries, the accuracy was 78% on IMDB, 65% on Yelp, and 72% on MAS. Finally, on extra-hard queries, G-SQL achieved 55% on IMDB, 45% on Yelp, and 50% on MAS., demonstrating robust performance across varying query complexities.

INDEX TERMS NL2SQL, natural language interface (NLI), text to SQL, NLIDB, schema-aware translation, semantic parsing, database query generation.

I. INTRODUCTION

Tody, data drives nearly every aspect of human activity—from commerce and government to education and scientific discovery [1]. Relational databases continue to be a dominant and efficient means of storing and managing structured data [2]. Accessing this data typically requires knowledge of Structured Query Language (SQL), a syntax-intensive language used to retrieve and manipulate relational data. Users of Database Management Systems (DBMSs) exhibit a wide range of expertise—from professional developers and data scientists to analysts unfamiliar with schema design, and non-expert users with little to no technical background. Constructing accurate SQL queries, particularly for complex questions, remains a significant challenge for many users [3].

The associate editor coordinating the review of this manuscript and approving it for publication was Ye Liu.

To address this accessibility gap, Natural Language to SQL (NL2SQL) systems have been developed. These systems enable users to express queries in natural language and have them automatically translated into executable SQL commands [4], [5]. As illustrated in Figure 1, the question "What year was Ellen Page born?" can be transformed into the SQL query: SELECT actor_0.birth_year FROM actor AS actor_0 WHERE actor_0.name = "Ellen Page". This transformation relies on an intermediate understanding of the database schema—often encoded in a structured format such as JSON—to effectively map linguistic elements to schema components. Despite the promise of NL2SQL systems, several key challenges persist [6], [7]:

- Fuzzy linguistic coverage: users may express the same query using varied phrasing.
- Linguistic vs. conceptual mismatches: parsing issues or poor schema understanding can cause system failure.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

- Language ambiguity: certain terms may map to multiple schema elements.
- Out-of-domain generalization: models may not operate effectively on unseen domains or schemas.

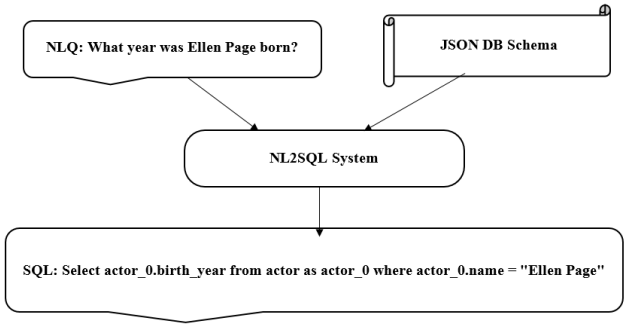

FIGURE 1. Conversion of natural language query to SQL Using a JSON-based schema.

Numerous studies have addressed the problem of translating NLQ into SQL. Some researchers employ encoder-decoder-based deep learning algorithms [8], [9], [10], [11], [12]. while others utilize rule-based algorithms [13], [14], [15], [16]. Although deep learning-based models have achieved remarkable results, they present several critical challenges [17]:

- They struggle with complex SQL constructs involving multiple joins, conditions, aggregations, and subqueries, often reducing execution accuracy to below 50%.
- They tend to generate non-executable SQL queries when treating NL2SQL as a pure sequence-to-sequence translation task.
- They lack schema awareness, leading to semantic mismatches.
- Effective training heavily requires relying on large annotated SQL-NL pairs, making scalability difficult.

These limitations highlight the enduring value of rule-based techniques, particularly in enhancing interpretability and robustness [17]. In this paper, we propose G-SQL (Guided SQL), a schema-guided rule-based system designed to overcome the aforementioned limitations. G-SQL focuses on two key components:

- Modeling the database schema through JSON serialization, which involves converting the schema into a structured JSON format to enable easy storage, interpretation, and automated processing.
- Analyzing user queries through structured syntactic and semantic processing.

To achieve this, G-SQL introduces a JSON-based schema representation that organizes database knowledge into four hierarchical levels: schema, tables, columns, and constraints. Each level carries semantic metadata to aid query interpretation. Furthermore, a domain-specific dictionary is incorporated to distinguish general linguistic synonyms from domain-specific terminology. NLQs are processed using the Stanford CoreNLP toolkit, which integrates rule-based, probabilistic, and deep learning methods [18]. Semantic matching is further enhanced through the use of pre-trained GloVe embeddings, supporting value-to-column and intent linking.

Although modern NL2SQL research increasingly explores large language models and end-to-end neural architectures, recent studies have demonstrated that classical NLP components such as part-of-speech tagging, dependency parsing, named entity recognition, and embedding models like GloVe continue to play an important role in state-of-the-art systems. These components are often used as complementary pre-processing steps that improve schema linking, interpretability, and query alignment before the final reasoning stage [68], [69] [81], [83]. In G-SQL, these modules are not presented as contributions by themselves but are deliberately integrated into the framework to support the proposed JSON-based schema serialization and rule-guided clause construction. This hybrid use of symbolic and linguistic features enables more robust and interpretable SQL generation, particularly in complex or low-resource scenarios where purely neural approaches frequently fail to produce executable queries.

The key contributions of this paper can be summarized as follows:

- Introducing a novel schema-aware NL2SQL system (G-SQL). Unlike purely neural models, G-SQL leverages a rule-based architecture enhanced by syntactic and semantic analysis tools to produce executable SQL queries from natural language input.
- Proposing a structured JSON-based serialization mechanism for representing database schemas in four hierarchical levels: database, tables, columns, and constraints. This format captures both structural and semantic information, including semantic names, frequent values, and key types (primary, foreign, datatype, etc.).
- Incorporating a domain dictionary (DoSyn) to bridge the gap between natural language expressions and schema terminology. It includes synonyms and type annotations, allowing for improved value-column mapping and schema awareness.
- Proposing a robust approach to query understanding and clause identification, enabling accurate extraction of grouping, aggregation, and join conditions. The system also distinguishes between direct and indirect table relationships in the schema.
- Integrating of Stanford CoreNLP and GloVe embeddings supports advanced parsing and semantic similarity detection without end-to-end neural training, improving interpretability and adaptability.
- Demonstrating performance on benchmark datasets (IMDB, Yelp, MAS), where G-SQL achieves high execution accuracy, especially in low- and medium-complexity queries, outperforming recent rule-based and deep learning-based approaches.
- Highlighting the potential use of G-SQL as a documentation and modeling layer in database design. JSON

schema and domain dictionary can support schema visualization, onboarding, and cross-domain knowledge transfer.

The remainder of this paper is structured as follows: Section II reviews the evolution of NL2SQL methods, highlighting key milestones and challenges. Section III describes the proposed G-SQL system in details, including schema modeling and query generation processes. Section IV presents experimental results and analysis. Finally, Section V concludes the paper and outlines directions for future work.

II. RELATED WORK

Undoubtedly, the vast number of domains linked to natural language interfaces for databases has driven many researchers to seek effective approaches to solve the NL2SQL problem [6]. This challenge has long captured the interest of scholars, dating back to early studies such as BASEBALL [19], LUNAR [20], RENDEZVOUS [21], LADDER [22], Chat-80 [23], ASK [24], and JANUS [25]. As the field progressed, two dominant paradigms emerged: rule-based approaches, which rely on handcrafted patterns and schema-driven logic, and deep learning-based approaches, which employ neural architectures to learn mappings from natural language to SQL. While rule-based systems offer transparency and structural control, they often lack flexibility and generalization capabilities. In contrast, neural models excel at adapting to linguistic variation but may generate syntactically incorrect or semantically invalid queries. The limitations inherent in both paradigms have led to the emergence of hybrid models, which seek to combine the interpretability and schema-awareness of rule-based systems with the adaptability and semantic richness of deep learning. These hybrid approaches represent a promising middle ground in the ongoing evolution of NL2SQL solutions.

A. RULE-BASED APPROACH

Between 2000 and 2018, numerous studies proposed rule-based solutions to address the NL2SQL challenge. Early algorithms like SODA [26], NLP-Reduce [27], SINA [28], and Précis [29] utilized techniques based on matching query keywords with the metadata's inverted index. Afterward, more sophisticated algorithms, such as QuestIO [30] and NLQ/A [31] replaced simple keyword matching with searches for linguistic or database-related patterns. Further enhancements were applied in handling semantic meaning by parsing NLQs, as seen in systems like ATHENA [15], ATHENA++ [16], Querix [32], USI Answers [33], Nalix [34], NaLIR [35], [36], and TEMPLAR [13]. Another approach emerged, focusing on developing rule-based frameworks that allowed users to construct their queries using a defined grammar. Examples of this trend include Ask Now [37], TR Discover [38], and Ginseng [39]. These algorithms offered users more flexibility while maintaining a rule-driven approach to query generation. These algorithms were improved through the use of word embedding techniques, including tf-idf [40], WordNet [41], Bag of Words (BOW) [42], Word2Vec [43], and GloVe [44] These methods enhanced the algorithms' ability to understand and process NLQs by providing richer semantic representations of words. Additionally, advancements in syntactic dependency parsing led to improved comprehension of NLQs [45]. Tools such as spacy [46], Stanford NLP Parser [47], NLTK [48], CoreNLP [18], and OpenNLP [49] enabled more effective analysis of the grammatical structure and dependencies in natural language sentences, which, in turn, led to more accurate NL2SQL query generation.

B. DEEP LEARNING APPROACH

The deep learning approach transforms the NL2SQL problem into a supervised learning problem by considering question-answer pairs. In these pairs, natural language questions are paired with corresponding SQL queries. The initial step involves converting these questions and SQL queries into vectors using word embedding techniques. Once vectorized, a deep neural network processes the data to generate SQL queries [50]. The deep learning method has two key components: the encoder and the decoder [51]. The encoder takes variable-shaped inputs and converts them into fixed internal representations, then passes these representations to the decoder, which predicts the most likely SQL query (or its parts) [52]. Both the encoder and decoder utilize various types of neural networks, such as recurrent neural networks (RNNs) [53], Long short-term memory (LSTMs) [54], Bidirectional Long Short-Term Memory (BILSTMs) [55], and Pretrained Language Models (PLMs) like Bidirectional Encoder Representations from Transformers (BERT) [56], Generative Pre-trained Transformer (GPT) [57], and self-attention mechanisms [58]. Examples of these deep learning-based algorithms include IR-Net [59], TypeSQL [60], Seq2SQL [61], and SyntaxSQLNet [62] using bidirectional LSTM, SQLova [63], and SLSQL [64] using PERT. Many deep learning algorithms treat the NL2SQL problem as a machine translation task, capitalizing on recent progress in this area. State-of-the-art algorithms like RATSQL [9] and SADGA [10] represent significant advances in treating NL2SQL as a machine translation problem, benefiting from developments in NLP and deep learning. However, deep learning-based solutions for NL2SQL face a critical limitation: they often fail to adequately consider the underlying database schema. This oversight leads to the generation of non-executable SQL queries. This issue was highlighted by the CATSQL method [17] CATSQL focuses on incorporating necessary details such as table names, column references, and conditions derived from the user's query. This approach emphasizes the importance of schema-awareness, which greatly improves the precision and reliability of the SQL output. Building upon this direction, several advanced models have recently emerged to further enhance semantic parsing performance in NL2SQL systems. MetaSQL [68] introduces a generate-then-rank framework that leverages meta-learning to enhance generalization across different SQL

query patterns. While this approach improves performance on unseen schemas, it still relies heavily on the diversity and quality of generated candidates, which may not always capture complex query structures accurately. E-SQL [69] enhances Text-to-SQL translation by incorporating entity-level linking between questions and database schemas. This integration improves contextual understanding, especially for entity-rich domains. However, E-SQL depends on the accuracy of entity recognition and linking modules, which can limit its robustness in noisy or ambiguous input scenarios.

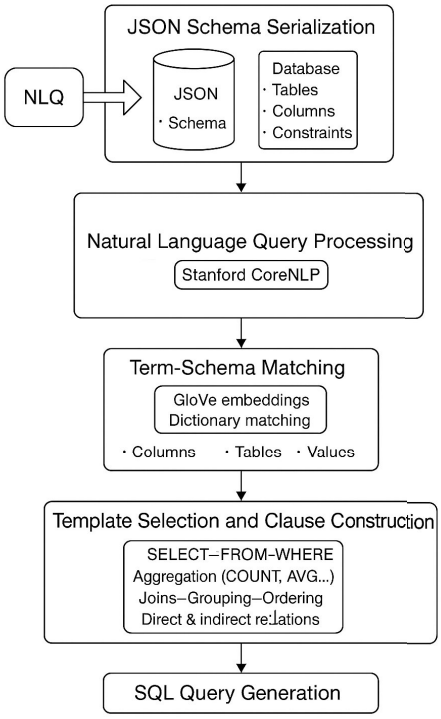

FIGURE 2. Overview of the GSQL architecture for converting natural language queries into SQL.

C. LLM-BASED APPROACHES

Recent advances in Large Language Models (LLMs) have significantly influenced the Text-to-SQL research landscape. Unlike traditional neural architectures that depend on extensive supervised training and fixed schema encoding, models such as GPT-3 [71], Codex [72], and ChatGPT [73] exhibit general-purpose text generation capabilities that can be adapted to semantic parsing with minimal fine-tuning. By leveraging zero-shot and few-shot prompting strategies, these models can produce SQL queries directly from natural language questions. Several systems, including SQL-Prompt [71], Text-to-SQL Bench [72], and Auto-CoT [73] employ in-context learning and chain-of-thought reasoning to achieve impressive results, often without explicit schema encoding.

While LLM-based solutions demonstrate strong adaptability and require little or no task-specific training data, they face important limitations that affect their reliability in real-world scenarios. Specifically, LLMs are prone to generating non-executable or hallucinated queries when schema structures become complex, due to their implicit rather than explicit schema understanding. They also require extensive computational resources and are sensitive to prompt engineering. Furthermore, the absence of explicit schema grounding makes it challenging to ensure syntactic correctness and semantic fidelity, particularly for highly specialized or unseen database schemas.

Recent evaluations have confirmed that deep learning-based NL2SQL approaches frequently produce non-executable or syntactically invalid SQL queries, especially when dealing with multi-table joins, nested subqueries, or unseen domains, due to limited schema grounding and lack of structural constraints during decoding [64], [78], [79]. Even advanced LLMs, including GPT-style models, continue to exhibit these weaknesses unless supported by additional self-correction or post-processing mechanisms [80], [81]. Consequently, hybrid methods that incorporate schema context into the prompt or combine LLMs with symbolic reasoning have been proposed to balance generalization with structural reliability.

In this context, we present Guided SQL (G-SQL), a schema-aware framework that integrates rule-based logic with deep learning techniques to translate NLQs into executable SQL statements. The core of the system is a structured representation of the database schema, encoded in a JSON document containing metadata such as table and column names, relational constraints, and key relationships. This explicit modeling enables G-SQL to generate syntactically correct and semantically meaningful SQL queries. While the overall architecture remains rule-driven, deep learning tools are strategically incorporated in specific components—such as value linking and query pattern recognition—to enhance semantic understanding and improve accuracy across varying levels of query complexity.

III. METHODOLOGY

This section provides an overview of the proposed framework before delving into the specifics of its design.

A. MODEL OVERVIEW

The process involves converting a NLQ into a structured SQL query to obtain useful information from the database. The NLQ, generally in English, contains key components:

- Table and Column Names: Tokens in the query refer to specific database tables and columns.
- Aggregation Functions: Operations such as sum, count, max, min, or average are applied to columns.
- Logical Expressions: Conditions and logical operators (e.g., AND, OR) are used to filter data.
- Semantic Words: Keywords that determine how tables are joined (e.g., INNER JOIN, LEFT JOIN).

Figure 2 illustrates the architecture of the G-SQL system, showcasing a structured and interpretable pipeline

for transforming NLQs into executable SQL statements. The system comprises five main stages, each representing a critical component in the translation process: schema serialization, natural language understanding, schema-term alignment, clause construction, and final SQL generation. A key strength of G-SQL lies in its explicit schema modeling using a structured JSON format. This enables the system to maintain a high level of transparency and adaptability across domains. Unlike black-box neural approaches, G-SQL separates the semantic and syntactic interpretation of queries from the structural knowledge of the database, providing greater control and explainability. Additionally, G-SQL benefits from the integration of Stanford CoreNLP and GloVe embeddings, which aid in understanding linguistic nuances and enhance the robustness of term-schema matching. The inclusion of a domain dictionary further improves the system's ability to resolve synonyms and ambiguous terms based on contextual meaning. Overall, G-SQL offers a modular and extensible architecture that supports accurate query translation and can be extended for schema documentation, query recommendation, and educational applications. Its rule-based foundation, complemented by deep learning tools, provides a hybrid yet interpretable solution to the NL2SQL problem.

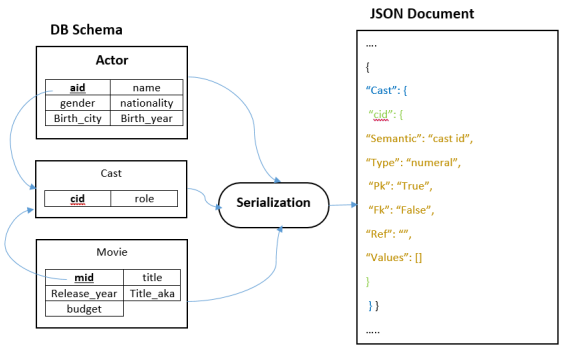

FIGURE 3. Database schema serialization into JSON format.

B. PROBLEM DEFINITION

Given a NLQ and a relational database schema (S), the objective is to generate a syntactically correct and semantically meaningful SQL query (Q) that retrieves the correct answer from the database. Formally:

- Let NLQ denote the natural language input expressed by the user.
- Let $S = T_1, T_2 \ldots T_n$ represent the schema of the relational database, where each table T_i contains a set of columns and associated constraints.
- Let M be a mapping function that aligns tokens in NLQ with elements in S (e.g., tables, columns, values).

The NL2SQL task involves constructing a function $f : (NLQ, S) \rightarrow Q$, such that the generated query Q satisfies:

- Syntactic validity: Q conforms to SQL grammar.
- Semantic correctness: Q reflects the user's intent as captured in NLQ.
- Executability: Q executes successfully on the target database and retrieves the correct results.

In this paper, a rule-based system is proposed, G-SQL, that approximates the function f by employing:

- A structured serialization of schema S into a JSON format.
- Syntactic and semantic parsing of NLQ using linguistic tools.
- Guided clause assembly based on predefined templates and schema-awareness.

This definition enables the problem to be solved without relying on large-scale training data, while maintaining transparency, adaptability, and robustness across domains.

C. SCHEMA MODELING AND JSON SERIALIZATION

Understanding the underlying database schema is a critical prerequisite for accurately interpreting user queries in NL2SQL systems, as emphasized by prior research [66]. In real-world settings, domain experts may conceptually understand the data but often lack the SQL expertise to construct correct queries. To address this challenge, G-SQL introduces an explicit and structured approach to schema modeling. A core component of this approach is a hierarchical JSON-based serialization that encodes both the structural and semantic components of the schema. This allows the system to effectively map elements of the NLQ to the appropriate database elements during translation. JSON serialization has been widely recognized as a practical and portable way to encode structured data in modern data-intensive systems [77]. The schema is organized into four hierarchical levels:

- Schema Level: Captures the overall structure of the database and serves as the root layer.
- Table Level: Contains all tables along with their semantic identifiers and relational metadata.
- Column Level: Describes the columns associated with each table, including column names and descriptive metadata.
- Column Description Level: Provides detailed annotations for each column, such as data types, primary and foreign keys, frequently used values, and flags indicating super columns (columns named after their parent tables).

1) DOMAIN DICTIONARY

A key challenge in NL2SQL is the semantic variability in user expressions—users often use synonyms or informal terms that do not match exact schema labels. To bridge this gap, G-SQL incorporates a domain-specific dictionary (DoSyn) directly into the JSON schema. This component enriches the system's understanding by offering:

- Synonyms for table and column names.
- Type annotations to indicate whether a term refers to a table, column, or value.
- Contextual meanings aligned with domain-specific usage.

By embedding this dictionary directly into the schema, G-SQL bridges the semantic gap between user intent and schema structure, significantly improving its robustness and accuracy in natural language understanding. Figure 3 depicts the layered structure of the serialized schema, while Figure 4 demonstrates an example of the domain dictionary and its application in resolving ambiguity during query interpretation.

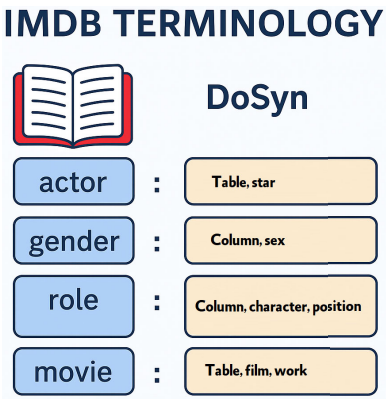

FIGURE 4. Domain dictionary (DoSyn) for IMDB terminology.

This investigation employed a manual approach for the JSON schema serialization process to guarantee the integrity and fidelity of the data model throughout the system's development lifecycle. Subsequent research endeavors will focus on the development of an automated ERD-to-JSON tool, with the objective of streamlining the schema generation workflow and mitigating the potential for human error.

2) WHY USE JSON SERIALIZATION FOR SCHEMA MODELING

While graph-based schema representations have traditionally been the dominant choice for modeling relational structures—owing to their ability to naturally capture entity relationships—they tend to be less interpretable, more complex to maintain, and harder to integrate into standardized data pipelines. In contrast, JSON schema serialization offers a structured, hierarchical, and human-readable representation that encodes database semantics in a portable format. This approach aligns with modern data engineering practices, facilitates schema-aware natural language processing, and simplifies value-column linking. Moreover, JSON serialization supports embedding additional semantic metadata (e.g., synonyms, frequent values, key constraints) that can be leveraged during query interpretation, while maintaining compatibility with automated pipelines and future extensions such as ERD-to-JSON conversion. Table 1 highlights the key differences between these two approaches.

D. NATURAL LANGUAGE QUERY PARSING AND ANNOTATIONS

Once the database structure is serialized into a JSON format, the next crucial step in the G-SQL pipeline is to process the NLQ. To achieve this, several NLP annotations are applied during the preliminary query analysis phase: Tokenization: The input query is segmented into discrete units called tokens, which may represent individual words or multi-word expressions. Tokenization forms the foundational step for subsequent linguistic analysis. Part-of-Speech (POS) Tagging: Each token is classified according to its grammatical category, such as noun, verb, adjective, or adverb. POS tagging enhances the system's ability to understand sentence structure and is critical for tasks like query parsing and entity recognition. For example, "NN" denotes a singular common noun, "NNS" a plural common noun, and "NP" a singular proper noun. Dependency Parsing: This step uncovers the syntactic structure of the sentence by analyzing grammatical relationships between tokens. Dependency parsing identifies the hierarchical links between words, such as subject-verb or object-preposition relationships, providing deeper insights into query semantics. Lemmatization: Words are reduced to their canonical dictionary forms by removing inflectional variations. Lemmatization ensures consistency during schema matching, allowing tokens from the user query to be accurately compared against schema elements and domain-specific entries.

FIGURE 5. Stanford CoreNLP annotations for natural language query processing in G-SQL.

These annotations for NLQs are obtained using the Stanford CoreNLP toolkit, developed by the Stanford Natural Language Processing Group [18]. Given that the toolkit's POS tagger reports an accuracy of 97%, the annotations are adopted directly without requiring additional post-processing.

E. G-SQL TAGS FOR SEMANTIC MAPPING

In every NLQ, specific keywords or phrases can be semantically aligned with components of the database schema—namely tables, attributes (columns), or values. To facilitate this alignment, G-SQL introduces a tagging mechanism that assigns semantic type labels (tags) to these tokens during preprocessing. These tags guide the construction of accurate

TABLE 1. Comparison between JSON serialization and graph-based schema representation.

Aspect	JSON Serialization	Graph-Based Representation
Interpretability	Simple hierarchical structure, easy to read and edit	Requires graph visualization tools for interpretation
Metadata Integration	Directly stores metadata (keys, synonyms, frequent values)	Limited direct embedding of metadata
Portability	Lightweight, standardized JSON text files	Requires specialized graph parsers/libraries
Support for Query Processing	Facilitates direct token-to-schema mapping	Strong for relationship-oriented reasoning
Integration with NLP	Naturally aligns with token-level semantic linking	Needs additional transformation for textual matching
Automation	Easily automated from ER diagrams to JSON	More complex automation workflows

and executable SQL queries. G-SQL defines six distinct tags, each corresponding to a specific role in query interpretation:

- Meta: Denotes metadata elements such as table or column names. These terms may appear in literal form (e.g., "movie," "title"), in modified variants (e.g., pluralized: "movies"), or as synonyms (e.g., "film," "player"). This tag supports schema-element matching during translation.
- Value: Represents actual data values that may appear in the database, such as "H. V. Jagadish" in the Name column of the Author table. These values are identified through CoreNLP parsing or derived from the serialized JSON schema.
- AGG (Aggregation): Marks tokens associated with aggregate functions such as SUM, AVG, COUNT, MAX, and MIN. Recognizing these tags helps the system to formulate correct aggregation clauses in SQL.
- Cond (Condition): Identifies tokens that signal filter conditions (e.g., "after 2010," "greater than 5"). These guide the formation of WHERE clauses and conditional logic in the query.
- Help (H): Tags auxiliary words that indirectly affect SQL structure. For instance, words like "per" or "each" may imply the need for GROUP BY, and negations may indicate outer joins or subqueries.
- Other (O): Includes words that are not mapped to any schema element and do not contribute directly to query logic. These are typically stop words (e.g., "the," "of," "a") and are ignored during SQL generation.

This tagging framework enhances the interpretability of the translation process, ensuring that each element of the query is handled appropriately according to its functional role in SQL generation.

F. SCHEMA LINKING

Schema linking is a crucial step in the NL2SQL pipeline, following the schema serialization process. Its primary goal is to establish connections between NLQ tokens and corresponding schema components, such as tables, columns, and values. The linking process involves four key subtasks: name linking, value linking, condition linking, and set linking. Name linking: This step aims to accurately identify references to table and column names within the user query. It leverages several strategies to align textual elements in the query with schema definitions:

- Direct match: When users explicitly mention table or column names, exact or partial string matching can be used.
- Linguistic variation: Prefixes, suffixes, or plural forms may alter the term. Lemmatization is applied to revert words to their base form for comparison.
- Synonymy: Users may use synonyms (e.g., "film" for "movie"). A domain-specific dictionary helps map these expressions to their respective schema elements.
- Super columns: Some users refer to a table using its name as a placeholder for a column, especially when the table has a representative attribute (e.g., "author" meaning "author.name").

Value linking: This process identifies value tokens in the query and determines the most appropriate column for each. It consists of two phases: Value detection, using:

- Value Store: Predefined common values stored during schema serialization.
- Named Entity Recognition (NER): Used to extract entities such as names, locations, or dates.
- Value Identification Rule: Any unmatched token that is not a stop word is considered a potential value.

Column association, using:

- Column Store: A dictionary of known column-value pairs to facilitate accurate matching.
- Super Value: If a value is used alongside a table name, it is likely linked to that table's super column.
- Head Token Analysis: Utilizing CoreNLP annotations to identify syntactic heads and dependencies for contextual linking.
- GloVe Embeddings: When other methods fail, GloVe is used to determine the most semantically compatible column based on vector similarity.

Condition linking: Once a value is linked to a column, the next step is to determine the appropriate operator (e.g., $=, >, <$). By default, equality is assumed unless another operator is indicated through CoreNLP analysis. Logical connectors such as AND and OR are also identified to form compound conditions in WHERE clauses.

Set linking: Some queries involve set operations such as UNION and INTERSECTION. These are typically inferred from conjunctions:

- "AND" implies intersection between values.
- "OR" implies a union of results.

This multi-stage linking mechanism ensures that user queries are interpreted accurately, even when phrased in ambiguous or informal ways. The integration of linguistic tools, schema metadata, and semantic embeddings provides G-SQL with strong capabilities in aligning natural language with structured database content. Table 2 outlines the query analysis process. First, multi-word terms like "Steven Spielberg" and "Directed by" are identified. Next, each term is tagged using six predefined tags. "Movie" and "Directed by" are Meta tags, while "Steven Spielberg" and "2006" are identified as actual values. The term "after" is recognized as a condition.

Name linking then identifies the table or column names for the Meta tags, such as the Movie and Directed by tables. Value linking follows, matching "Steven Spielberg" to the Director.name column and "2006" to the release_year column. Finally, the conditions are set as "Director.name = Steven Spielberg" and "Movie.release_year > 2006".

G. SCHEMA-AWARE QUERY GENERATION

Rule-based NL2SQL systems retain several enduring advantages that make them relevant even in the era of deep learning and large language models (LLMs). Unlike purely neural approaches, rule-guided methods explicitly leverage the database schema and metadata to ensure that generated queries remain executable and semantically correct, thereby avoiding a common issue in deep learning–based models where queries often fail due to missing constraints or invalid joins [9], [17] [59] [78], [80] [81], [82]. These systems offer interpretability and explainability, since each translation step follows explicit, human-understandable patterns, making it possible to trace how natural language tokens are mapped to schema elements [15], [35]. Their deterministic behavior guarantees consistency across similar inputs, which is particularly valuable in mission-critical domains such as healthcare, finance, and legal analytics [16]. Moreover, they exhibit strong schema-awareness, ensuring that table relationships, keys, and value constraints are respected throughout the generation pipeline. Another advantage is their effectiveness in low-resource domains, as rule-based systems do not require large annotated corpora for training and can be adapted to new databases with minimal manual effort [16], [81]. Finally, these systems can be hybridized with machine learning components—such as embeddings, NER, or transformer-based semantic similarity—to handle complex linguistic variability while preserving structural control [9], [59]. In contrast, deep learning and LLM-based models excel at generalization but often rely on large-scale training data and may produce non-executable or semantically invalid SQL queries when schema constraints are not explicitly encoded [81]. These characteristics justify the adoption of schema-aware, rule-guided approaches such as G-SQL, which combine deterministic schema modeling with data-driven enhancements to achieve a balance between robustness, interpretability, and adaptability.

Algorithm 1 G-SQL: Guided Translation From Natural Language to SQL

```
Require: NLQ, JSONSchema: Structured DB schema (Tables, Columns, Constraints, Domain Dictionary)
Ensure: SQLQuery: Executable SQL query
// Step 1: Serialization
JSONSchema ← ParseDatabaseSchema(DB)
for all table T in DB do
    Add T to JSONSchema
    for all column C in T do
        Add ColumnInfo(C.name, datatype, PK/FK, values, isSuperColumn) to T
    end for
end for
DoSyn ← BuildDomainDictionary()
// Step 2: NLP Annotations using Stanford CoreNLP
Tokens ← Tokenize(NLQ)
Lemmas ← Lemmatize(Tokens)
POSTags ← POS_Tagging(Tokens)
Dependencies ← Dependency_Parse(NLQ)
// Step 3: Semantic Tagging
for all token in Tokens do
    Tag[token] ← AssignGSQLTag(token, JSONSchema, DoSyn)
end for
// Step 4: Value Linking using CoreNLP + GloVe
for all token in Tokens do
    if Tag[token] == "Value" then
        Column ← MatchValueToColumn(token, JSONSchema)
        if Column == NULL then
            Column ← GloVe_Match(token, JSONSchema.Columns)
        end if
        AddCondition(Column, token)
    end if
end for
// Step 5: Name Linking
for all token in Tokens do
    if Tag[token] == "Meta" then
        Matched ← MatchUsingDoSyn(token, JSONSchema)
        AddToQueryStructure(Matched)
    end if
end for=0
// Step 6: Condition Linking
Operators ← DetectOperators(Dependencies)
Conditions ← AssembleConditions(Tokens, Operators)
// Step 7: Clause Assembly
SelectClause ← DetermineSelectFields()
FromClause ← IdentifyRelevantTables()
JoinPaths ← InferJoinPaths(FromClause, JSONSchema)
WhereClause ← CombineConditions(Conditions)
GroupBy ← DetectGroupByHints(Tokens)
Having ← ExtractHavingClauseIfPresent()
// Step 8: SQL Generation
SQLQuery ← FormatSQL(SelectClause, FromClause, JoinPaths, WhereClause, GroupBy, Having)
return SQLQuery=0
```

TABLE 2. NLQ analysis example.

Tokens	Find	all	movies	Directed by	Steven Spielberg	after	2006
Multi-word	Find	All	Movies	Directed by	Steven Spielberg	after	2006
Type Tag	O	O	Meta	Meta	Value	Cond	Value
Name Linking			Movie	Directed by			
Value Linking					$Director.Name$	$Movie.release_year$	
Condition Linking				$Director.Name = StevenSpielberg$		$Movie.release_year > 2006$	

H. RULE CONSTRUCTION AND INTEGRATION

A core component of G-SQL is its hybrid rule-based layer, which combines the structured flexibility of dynamic rule generation with the clarity of well-organized rule categories. Unlike purely neural models that depend entirely on learned patterns, G-SQL automatically derives most of its rules at runtime using schema serialization and CoreNLP analysis, while maintaining a modular design that can be extended when adapting to new domains. The rule engine operates through three synergistic categories. Schema-aware rules, generated directly from the JSON-serialized schema, ensure that user tokens referring to tables and columns are accurately resolved even when synonyms, plural forms, or super-key attributes are used; the join paths between tables are inferred automatically from the schema constraints. Syntactic rules exploit CoreNLP's dependency parsing, POS tagging, and entity recognition to capture relationships between tokens, enabling the automatic detection of conditions, operators, aggregations, and grouping constructs from natural language patterns such as "count of X after Y." Semantic expansion rules enhance robustness through the use of domain dictionaries and GloVe embeddings, allowing the system to link informal or ambiguous terms with schema elements based on meaning rather than surface forms. By unifying these rule types, the framework constructs intermediate representations that dynamically guide SQL generation—including table and column selection, value linking, condition formulation, and join inference—without requiring handcrafted query templates. At the same time, its modular architecture allows the incorporation of additional domain-specific rules when necessary. This hybrid mechanism gives G-SQL the benefits of interpretability, scalability, and adaptability across domains while retaining the semantic flexibility of neural components.

IV. EXPERIMENTAL EVALUATION

A. DATASETS AND EVALUATION METRICS

In our experiments, we utilized the IMDB [70], yelp [67] and MAS [14] datasets, which are commonly used by the database community for NL2SQL-related research. IMDB (Internet Movie Database) is an online platform that offers extensive data on films, television shows, video games, and various industry professionals such as actors, directors, and producers. It serves as a comprehensive resource for information about plot summaries, cast and crew details, ratings, and production details across the entertainment industry. The Yelp Dataset is an essential resource for research, teaching, and learning, offering a comprehensive collection of real-world data related to businesses, customer reviews, and user interactions. It includes data on local businesses, detailed user reviews, and ratings, which researchers can use to explore various topics like NLP, machine learning, recommendation systems, and sentiment analysis. This dataset provides valuable insights into consumer behavior and business performance across multiple industries. The Microsoft Academic Search (MAS) database, created by Li et al., is derived by enumerating every logical query that could be expressed using the search page of the MAS website and writing sentences to match them, in the English language. When evaluating an NL2SQL's performance, two critical factors are considered: response accuracy and ease of use for the end user. One commonly used method to measure performance is Execution Accuracy (EX) [51]. Execution accuracy involves comparing the results of the actual SQL query to those of the predicted SQL query generated by the NL2SQL. If the outputs match exactly, the predicted query is considered correct. This approach emphasizes not just syntactical accuracy, but also the actual relevance and correctness of the query execution.

B. EVALUATION RESULTS

We evaluated the G-SQL algorithm on the IMDB, Yelp, and MAS datasets, which contain 131, 128, and 196 human-generated queries, respectively. The queries vary in complexity, as detailed in Table 3. These datasets serve as benchmarks to assess G-SQL's performance across diverse SQL constructs, including grouping, aggregation, joins, and complex conditional expressions, thereby evaluating its ability to handle varying levels of query complexity.

TABLE 3. Statistics for NL2SQL benchmarks.

	Tables	Columns	Queries	Easy	Medium	Hard	Extra- Hard
IMDB	16	65	131	23	13	67	28
Yelp	7	38	128	12	57	27	32
MAS	17	42	196	20	29	25	113

To gain a deeper understanding of G-SQL's behavior and robustness, we conducted a fine-grained evaluation by categorizing the test queries into four complexity levels: easy, medium, hard, and extra-hard. This categorization, based on the Spider benchmark classification, reflects increasing syntactic and semantic difficulty in translating NLQs into executable SQL statements. As shown in Table 4, G-SQL achieved 100% execution accuracy on all easy queries across the IMDB, Yelp, and MAS datasets. These queries typically

involve straightforward SELECT-FROM-WHERE structures with single-table references and clearly identifiable schema elements. The accurate query construction and template-based SQL generation in these instances can be attributed to the use of a structured JSON schema representation, combined with guided linking strategies.

TABLE 4. Execution accuracy for IMDB, Yelp and MAS.

	IMDB		Yelp		MAS	
	#Q	EX	#Q	EX	#Q	EX
Easy	23	1.00	12	1.00	20	1.00
Medium	13	0.95	57	0.85	29	0.90
Hard	67	0.78	27	0.65	25	0.72
Extra- Hard	28	0.55	32	0.45	113	0.5

For medium-complexity queries, G-SQL maintained high accuracy, particularly on the IMDB and MAS datasets. These queries commonly involve multiple fields or moderate filtering logic, and the semantic clarity of schema terminology in these datasets facilitates effective mapping of columns and values. However, a slight decrease in performance was observed on the Yelp dataset, primarily due to ambiguities in query phrasing and less clearly defined schema relationships. For example, distinguishing between similar terms such as "review score" and "business rating" occasionally led to incorrect mappings. As expected, G-SQL's accuracy decreased with increasing query complexity. Hard and extra-hard queries typically involve multi-table joins, nested SELECT statements, aggregation functions (e.g., COUNT, AVG), and implicit or contextual conditions. The system demonstrated stronger performance on the MAS dataset in these categories, likely benefiting from its more consistent and organized schema design. In contrast, the Yelp dataset posed greater challenges due to its more flexible and less structured schema and increased linguistic variability in user queries. The most frequent sources of errors included the inability to process nested subqueries and aggregation combined with conditional filtering (e.g., "Find actors who appeared in more than 5 movies after 2010"). These limitations highlight areas for future development, as G-SQL currently does not support the full range of advanced SQL features. In summary, the results demonstrate that G-SQL exhibits robust and consistent performance in handling low- to medium-complexity queries. It also shows promising adaptability to more complex scenarios, especially when the underlying database schema is well-structured and query intent is explicit. Future work will focus on enhancing the system by incorporating support for nested logic, advanced aggregations, and improved multi-table reasoning to further improve performance in high-complexity query environments.

Figure 6 illustrates the execution accuracy (EX) of the proposed G-SQL system in comparison with two well-known NL2SQL baselines—CatSQL [65] and IRNet [59]—across three benchmark datasets: IMDB, Yelp, and MAS. The results demonstrate that G-SQL consistently outperforms both CatSQL and IRNet across all datasets, highlighting

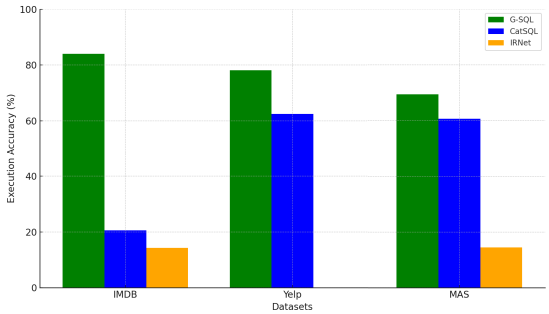

FIGURE 6. Execution accuracy comparison of G-SQL, CatSQL, and IRNet on three benchmark datasets (IMDB, Yelp, and MAS).

its robustness in handling varied query complexities and schema structures. The evaluation is conducted using the execution accuracy (EX) metric, a widely adopted and reliable criterion for assessing the effectiveness of natural language to SQL translation systems. Execution accuracy quantifies the correctness of the predicted SQL queries by comparing their output with that of the corresponding ground-truth SQL queries when executed over the same database content. It is formally defined as:

$$Score(V, \hat{V}) = \begin{cases} 1 & V = \hat{V} \\ 0 & V = \hat{V} \end{cases} \tag{1}$$

$$EX = \frac{\sum_{n=1}^{N} Score(Y, \hat{Y})}{N}. \tag{2}$$

On the IMDB dataset, G-SQL achieved the highest execution accuracy at 84%, significantly outperforming CatSQL (20.61%) and IRNet (14.29%). This substantial performance margin highlights the strength of G-SQL's schema-aware strategy and its effectiveness in structured and semantically coherent environments. In the Yelp dataset, which presents more ambiguity in schema structure and user query formulation, G-SQL maintained a strong lead with 78.1% accuracy. CatSQL followed with 62.5%, while IRNet did not yield successful results, suggesting difficulties in generalizing to less structured and semantically noisy schemas. The robustness of G-SQL in this context can be attributed to its explicit schema modeling through JSON serialization and its domain dictionary.

For the MAS dataset, G-SQL again led with 69.4% execution accuracy, compared to 60.71% for CatSQL and 14.52% for IRNet. The MAS dataset's academic domain, featuring abstract relationships among entities, further demonstrates G-SQL's ability to generalize through guided schema linking and structured clause generation, where IRNet struggled with complex logic and entity disambiguation.

Table 5 presents qualitative examples from the IMDB, Yelp, and MAS datasets, demonstrating the proposed G-SQL system's ability to generate syntactically and semantically accurate SQL queries in response to NLQs. Each example provides a direct comparison between the ground-truth SQL

query and the corresponding query generated by G-SQL. In the IMDB example, the system is tasked with retrieving the name of the actor who played a specific role in a particular movie. G-SQL accurately identifies and maps key semantic elements, such as "Captain Miller" (role), "Saving Private Ryan" (movie title), and the appropriate relational joins between the actor, cast, and movie tables. The generated SQL query matches the ground-truth query, confirming the system's ability to handle multi-table joins and role-based constraints with precision. The Yelp query involves a simple aggregation task: counting users named "Michelle." In this case, G-SQL correctly recognizes the entity ("user"), the filtering condition (name = "Michelle"), and the appropriate aggregation function (count(distinct name)). The alignment with the ground-truth query demonstrates the system's effectiveness in handling name-based filtering and aggregation over single-table queries. The MAS dataset example illustrates a more complex query involving grouping and aggregation by publication year for a given author. G-SQL accurately identifies the relevant entities (author, publication, writes) and constructs the necessary join conditions. Notably, the system correctly reproduces the group-by clause, enabling the accurate counting of publications per year. Despite the query's complexity, the generated SQL matches the ground-truth query both structurally and semantically. Collectively, these examples underscore G-SQL's robustness and accuracy across a range of query patterns, from simple selections to complex multi-table aggregations and join conditions. The model's ability to align with the ground-truth in diverse scenarios reinforces its effectiveness in schema-aware translation and structured query generation.

C. ERROR ANALYSIS AND LIMITATIONS

A focused error analysis across the IMDB, Yelp, and MAS datasets reveals that most of the residual errors in G-SQL are caused by limitations inherent to rule-based parsing rather than join inference. The primary source of errors in G-SQL arises from its difficulty in handling complex linguistic constructions and deeply nested subqueries. These challenges occur when user queries require implicit reasoning, such as inner aggregations or ranking steps, that are not explicitly stated through clear keywords like "GROUP BY" or "AVERAGE." In such cases, the rule-based system often generates incomplete or incorrect SQL structures. Additional errors may result from ambiguous synonyms, vague temporal or descriptive expressions, and requests for data not represented in the schema. To address these limitations, future work will focus on integrating Large Language Models (LLMs) into the G-SQL framework. LLMs can enhance the system's ability to interpret complex and context-dependent language, dynamically generate nested query structures, and improve overall robustness while maintaining G-SQL's schema-aware and rule-guided foundation. Importantly, multi-table join errors are rare thanks to the schema serialization and rule-guided assembly, which handle table relations effectively.

An illustrative example is the query: "List the top three directors who, after 2015, have made movies in genres that were not among the top five genres in terms of average revenue before 2010."

For this query, the current G-SQL system generates:

```
SELECT d.name, COUNT(m.movie_id) AS movie_count
FROM directors d
JOIN movies m ON d.director_id = m.director_id
JOIN genres g ON m.genre_id = g.genre_id
WHERE m.release_year > 2015
GROUP BY d.name
ORDER BY movie_count DESC
LIMIT 3;
```

While this output correctly handles table selection, filtering, and counting, it fails to produce the nested subquery required to exclude the top five genres by average revenue before 2010. The correct SQL should be:

```
SELECT d.name, COUNT(m.movie_id) AS movie_count
FROM directors d
JOIN movies m ON d.director_id = m.director_id
JOIN genres g ON m.genre_id = g.genre_id
WHERE m.release_year > 2015
AND g.genre_id NOT IN (
SELECT g2.genre_id
FROM movies m2
JOIN genres g2 ON m2.genre_id = g2.genre_id
WHERE m2.release_year < 2010
GROUP BY g2.genre_id
ORDER BY AVG(m2.revenue) DESC
LIMIT 5
)
GROUP BY d.name
ORDER BY movie_count DESC
LIMIT 3;
```

This failure occurs because the phrase "genres that were not among the top five genres in terms of average revenue before 2010" implies an inner aggregation and ranking step without explicitly stating keywords like "group by" or "average." Such implicit reasoning is beyond the capabilities of a purely rule-based system, which depends on explicit patterns. These types of queries, involving deeply nested conditions or implied subqueries, remain a key source of errors and represent an important direction for future enhancements to G-SQL.

In addition to these two dominant error categories, several less frequent but noteworthy error types were also observed. Value linking errors occur when a user query contains vague temporal or descriptive expressions that do not directly match database values. For example, in the query "Find movies from the modern era," the term "modern era" is not explicitly stored in the schema and may not be correctly translated to a condition such as release_year > 2000. Synonym ambiguity errors can arise when a term corresponds to multiple possible schema elements. For instance, the word "rating" could refer to a movie rating or a director rating

TABLE 5. Queries samples for three datasets (IMDB, Yelp, and MAS).

NLQ	Find the actor who played Captain Miller in the movie Saving Private Ryan.	Find the number of users called Michelle.	Return me number of papers written by H. V. Jagadish in per year.
Ground-truth	select actor_0.name from actor as actor_0, cast as cast_0, movie as movie_0 where cast_0.role = "Captain Miller" and movie_0.title = "Saving Private Ryan" and actor_0.aid = cast_0.aid and cast_0.msid = movie_0.mid ;	select count(distinct user_0.name) from user as user_0 where user_0.name = "Michelle" ;	select publication_0.year, count (distinct publication_0.title) from author as author_0 join publication as publication_0 join writes as writes_0 where author_0.name = "H. V. Jagadish" and author_0.aid = writes_0.aid and publication_0.pid = writes_0.pid group by publication_0.year;
G-SQL	select actor_0.name from actor as actor_0, cast as cast_0, movie as movie_0 where cast_0.role = "Captain Miller" and movie_0.title = "Saving Private Ryan" and actor_0.aid = cast_0.aid and cast_0.msid = movie_0.mid;	select count(distinct user_0.name) from user as user_0 where user_0.name = "Michelle";	select publication_0.year, count (distinct publication_0.title) from publication as publication_0 join writes as writes_0 join author as author_0 where author_0.name = "H. V. Jagadish" and author_0.aid = writes_0.aid and publication_0.pid = writes_0.pid group by publication_0.year;

if both exist, and a purely rule-based approach may select the wrong column. Schema coverage errors happen when a user requests information that is not represented in the database (e.g., "Find directors' social media accounts"), which leads to incomplete or invalid SQL generation. Finally, uncommon aggregation formulations can cause failures when the intended operation is expressed in a form that is not captured by existing rule patterns. For example, a query such as "Which director works with the greatest diversity of genres?" implies a COUNT(DISTINCT genre) operation, which may not be matched by predefined templates. While these cases are less frequent, they can be addressed through incremental enhancements to the rule library and by enriching the domain synonym dictionary (DoSyn) to capture a broader range of value expressions and context-sensitive mappings.

D. CASE STUDY: APPLYING G-SQL TO SPIDER DATABASES

To demonstrate the generality of G-SQL beyond the single-domain datasets used in our main experiments, we conducted a small-scale qualitative case study on several databases from the Spider benchmark. Rather than reporting accuracy scores, this analysis focuses on how G-SQL interprets natural language queries and generates SQL statements by leveraging JSON schema serialization and its rule-based pipeline.

Example 1 – College Database

NLQ: "List the names of students and their advisors in the Computer Science department who have a GPA above 3.5." Using the schema serialization, G-SQL identifies the student and advisor tables, links advisor_id as the join key, and extracts the conditions from the query.

The resulting SQL is:

SELECT student.name, advisor.name
FROM student
JOIN advisor ON student.advisor_id = advisor.advisor_id
WHERE advisor.department = "Computer Science"
AND student.gpa > 3.5;

Example 2 – Flight_2 Database

NLQ: "Show the flight id, airline, and departure city for flights that departed after 2020." From the JSON schema, the system matches the relevant columns in flight and airline tables, uses airline_id to infer the join path, and applies the temporal filter:

SELECT flight.flight_id, airline.name, flight.departure_city
FROM flight
JOIN airline ON flight.airline_id = airline.airline_id
WHERE flight.departure_year > 2020;

Example 3 – Academic Database

NLQ: "List the titles of papers written by authors from MIT in 2022." G-SQL links author, writes, and paper tables using their foreign keys and constructs the filtering conditions:

SELECT paper.title
FROM author
JOIN writes ON author.author_id = writes.author_id
JOIN paper ON writes.paper_id = paper.paper_id
WHERE author.affiliation = "MIT"
AND paper.year = 2022;

These examples highlight that, when the schema is serialized into JSON and the relationships between tables are explicit, G-SQL can successfully generalize to new database schemas without retraining. The results also confirm the strength of rule-guided join inference and value linking, while emphasizing that the main limitations occur with deeply nested or semantically implicit queries—an area we identify as a priority for future work.

V. CONCLUSION AND FUTURE WORK

In this paper, we presented G-SQL, a novel approach to the Natural Language to SQL (NL2SQL) challenge. G-SQL effectively integrates a rule-based methodology with deep learning techniques to address key aspects of the translation process. Furthermore, it leverages a JSON document-

based method for encoding database schema. Evaluation on benchmark datasets, including IMDB, Yelp, and MAS, demonstrated that G-SQL outperforms existing NL2SQL systems. G-SQL exhibited strong proficiency in handling name-based and value-based linking, and comparative analyses against several algorithms confirmed its effectiveness across datasets of varying SQL complexities, achieving competitive results. Further research should prioritize the refinement of the word embedding mechanism to achieve more accurate identification of relevant database columns for user-provided values. Moreover, the integration of Pre-trained Language Models (PLMs), such as BERT, GPT, or other enhanced models presents a significant opportunity to overcome existing limitations within G-SQL concerning the recognition of underlying conceptual relationships expressed in NLQs. In a related vein, a compelling avenue for future investigation lies in the automation of the schema serialization process. Currently, the construction of the JSON schema employed by G-SQL is performed manually. Automating this critical stage—through the development of a specialized tool capable of directly transforming Entity-Relationship Diagrams (ERDs) into the structured JSON format required by G-SQL—would substantially enhance both the scalability and usability of the system. Such automation would facilitate seamless integration with established database design methodologies and significantly reduce the temporal and resource demands associated with deploying Natural Language to SQL (NL2SQL) systems across diverse domains.

REFERENCES

[1] R. Kitchin, *The Data Revolution: Big Data, Open Data, Data Infrastructures and Their Consequences*. Newbury Park, CA, USA: Sage, 2014.

[2] F. Őzcan, A. Quamar, J. Sen, C. Lei, and V. Efthymiou, "State of the art and open challenges in natural language interfacesdoi to data," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Jun. 2020, pp. 2629–2636.

[3] A. Chopra and R. Azam, "Enhancing natural language query to SQL query generation through classification-based table selection," in *Proc. Int. Conf. Eng. Appl. Neural Netw.*, 2024, pp. 152–165.

[4] T. Yu, Z. Li, Z. Zhang, R. Zhang, and D. Radev, "TypeSQL: Knowledge-based type-aware neural text-to-SQL generation," 2018, *arXiv:1804.09769*.

[5] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, 2023, pp. 13067–13075.

[6] K. Affolter, K. Stockinger, and A. Bernstein, "A comparative survey of recent natural language interfaces for databases," *VLDB J.*, vol. 28, no. 5, pp. 793–819, Oct. 2019.

[7] R. C. A. Iacob, F. Brad, E.-S. Apostol, I. A. Hosu, and T. Rebedea, "Neural approaches for natural language interfaces to databases: A survey," in *Proc. 28th Int. Conf. Comput. Linguistics*, 2020, pp. 381–395.

[8] J. Qi, J. Tang, Z. He, X. Wan, Y. Cheng, C. Zhou, X. Wang, Q. Zhang, and Z. Lin, "RASAT: Integrating relational structures into pretrained Seq2Seq model for text-to-SQL," 2022, *arXiv:2205.06983*.

[9] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," 2019, *arXiv:1911.04942*.

[10] R. Cai, J. Yuan, B. Xu, and Z. Hao, "SADGA: Structure-aware dual graph aggregation network for text-to-SQL," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 7664–7676.

[11] U. Brunner and K. Stockinger, "ValueNet: A natural language-to-SQL system that learns from database information," in *Proc. IEEE 37th Int. Conf. Data Eng. (ICDE)*, Apr. 2021, pp. 2177–2182.

[12] B. Hui, X. Shi, R. Geng, B. Li, Y. Li, J. Sun, and X. Zhu, "Improving text-to-SQL with schema dependency learning," 2021, *arXiv:2103.04399*.

[13] C. Baik, H. V. Jagadish, and Y. Li, "Bridging the semantic gap with SQL query logs in natural language interfaces to databases," in *Proc. IEEE 35th Int. Conf. Data Eng. (ICDE)*, Apr. 2019, pp. 374–385.

[14] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," *Proc. VLDB Endowment*, vol. 8, no. 1, pp. 73–84, Sep. 2014.

[15] D. Saha, A. Floratou, K. Sankaranarayanan, U. F. Minhas, A. R. Mittal, and F. Özcan, "ATHENA: An ontology-driven system for natural language querying over relational data stores," *Proc. VLDB Endowment*, vol. 9, no. 12, pp. 1209–1220, Aug. 2016.

[16] J. Sen, C. Lei, A. Quamar, F. Özcan, V. Efthymiou, A. Dalmia, G. Stager, A. Mittal, D. Saha, and K. Sankaranarayanan, "ATHENA++: Natural language querying for complex nested SQL queries," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 2747–2759, Aug. 2020.

[17] H. Fu, C. Liu, B. Wu, F. Li, J. Tan, and J. Sun, "CatSQL : Towards real world natural language to SQL applications," *Proc. VLDB Endowment*, vol. 16, no. 6, pp. 1534–1547, Feb. 2023.

[18] C. Manning, M. Surdeanu, J. Bauer, J. Finkel, S. Bethard, and D. McClosky, "The Stanford CoreNLP natural language processing toolkit," in *Proc. 52nd Annu. Meeting Assoc. Comput. Linguistics: Syst. Demonstrations*, 2014, pp. 55–60.

[19] B. F. Green Jr., A. K. Wolf, C. Chomsky, and K. Laughery, "Baseball: An automatic question-answerer," in *Proc. IRE-AIEE-ACM Comput. Conf.*, 1961, pp. 219–224.

[20] W. A. Woods, "Progress in natural language understanding: An application to lunar geology," in *Proc. Nat. Comput. Conf. Expo.*, 1973, pp. 441–450.

[21] E. F. Codd, *Seven Steps to Rendezvous With the Casual User*, vol. 1333. Armonk, NY, USA: IBM Corporation, 1974.

[22] G. G. Hendrix, E. D. Sacerdoti, D. Sagalowicz, and J. Slocum, "Developing a natural language interface to complex data," *ACM Trans. Database Syst.*, vol. 3, no. 2, pp. 105–147, Jun. 1978.

[23] D. H. Warren and F. C. Pereira, "An efficient for interpreting easily adaptable system natural language queries," *Amer. J. Comput. Linguistics*, vol. 8, nos. 3–4, pp. 110–122, 1982.

[24] B. H. Thompson and F. B. Thompson, "Introducing ask, a simple knowledgeable system," in *Proc. 1st Conf. Appl. natural Lang. Process.*, 1983, pp. 17–24.

[25] R. M. Weischedel, "A hybrid approach to representation in the JANUS natural language processor," in *Proc. 27th Annu. Meeting Assoc. Comput. Linguistics*, 1989, pp. 193–202.

[26] L. Blunschi, C. Jossen, D. Kossman, M. Mori, and K. Stockinger, "SODA: Generating SQL for business users," 2012, *arXiv:1207.0134*.

[27] E. Kaufmann, A. Bernstein, and L. Fischer, "NLP-Reduce: A Naive but domain-independent natural language interface for querying ontologies," in *Proc. 4th Eur. Semantic Web Conf. (ESWC)*, vol. 7, 2007, pp. 1–2.

[28] S. Shekarpour, E. Marx, A.-C. N. Ngomo, and S. Auer, "SINA: Semantic interpretation of user queries for question answering on interlinked data," *J. Web Semantics*, vol. 30, pp. 39–51, Jan. 2015.

[29] A. Simitsis, G. Koutrika, and Y. Ioannidis, "Précis: From unstructured keywords as queries to structured databases as answers," *VLDB J.*, vol. 17, no. 1, pp. 117–149, Nov. 2008.

[30] D. Damljanovic, V. Tablan, and K. Bontcheva, "A text-based query interface to OWL ontologies," in *Proc. LREC*, 2008, pp. 1–8.

[31] W. Zheng, H. Cheng, L. Zou, J. X. Yu, and K. Zhao, "Natural language question/answering: Let users talk with the knowledge graph," in *Proc. ACM Conf. Inf. Knowl. Manage.*, Nov. 2017, pp. 217–226.

[32] E. Kaufmann, A. Bernstein, and R. Zumstein, "Querix: A natural language interface to query ontologies based on clarification dialogs," in *Proc. 5th Int. Semantic Web Conf. (ISWC)*, 2006, pp. 980–981.

[33] U. Waltinger, D. Tecuci, M. Olteanu, V. Mocanu, and S. Sullivan, "USI answers: Natural language question answering over (semi-) structured industry data," in *Proc. AAAI Conf. Artif. Intell.*, vol. 27, 2013, pp. 1471–1478.

[34] Y. Li, H. Yang, and H. V. Jagadish, "NaLIX: A generic natural language search environment for XML data," *ACM Trans. Database Syst.*, vol. 32, no. 4, p. 30, Nov. 2007.

[35] F. Li and H. V. Jagadish, "NaLIR: An interactive natural language interface for querying relational databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Jun. 2014, pp. 709–712.

[36] M. Dubey, S. Dasgupta, A. Sharma, K. Höffner, and J. Lehmann, "Asknow: A framework for natural language query formalization in SPARQL," in *Proc. Semantic Web. Latest Advances New Domains, 13th Int. Conf. (ESWC)*, Heraklion, Crete, Greece, 2016, pp. 300–316.

[37] D. Song, F. Schilder, C. Smiley, C. Brew, T. Zielund, H. Bretz, R. Martin, C. Dale, J. Duprey, T. Miller, and J. Harrison, "TR discover: A natural language interface for querying and analyzing interlinked datasets," in *Proc. 14th Int. Semantic Web Conf.*, Bethlehem, PA, USA, 2015, pp. 21–37.

[38] A. Bernstein, E. Kaufmann, and C. Kaiser, "Querying the semantic web with ginseng: A guided input natural language search engine," in *Proc. 15th Workshop Inf. Technol. Syst.*, Las Vegas, NV, USA, 2005, pp. 112–126.

[39] K. S. Jones, "A statistical interpretation of term specificity and its application in retrieval," *J. Documentation*, vol. 28, no. 1, pp. 11–21, Oct. 1972.

[40] G. A. Miller, "WordNet: A lexical database for English," *Commun. ACM*, vol. 38, no. 11, pp. 39–41, 1995.

[41] W. A. Qader, M. M. Ameen, and B. I. Ahmed, "An overview of bag of words; importance, implementation, applications, and challenges," in *Proc. Int. Eng. Conf. (IEC)*, Jun. 2019, pp. 200–204.

[42] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.

[43] J. Pennington, R. Socher, and C. Manning, "Glove: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.

[44] R. Frank, *Phrase Structure Composition and Syntactic Dependencies*, vol. 38. Cambridge, MA, USA: MIT Press, 2004.

[45] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," vol. 7, no. 1, pp. 411–420, 2017.

[46] M.-C. De Marneffe, B. MacCartney, and C. D. Manning, "Generating typed dependency parses from phrase structure parses," in *Proc. LREC*, vol. 6, 2006, pp. 449–454.

[47] S. Bird, E. Klein, and E. Loper, *Natural Language Processing With Python: Analyzing Text With the Natural Language Toolkit*. Sebastopol, CA, USA: O'Reilly Media, 2009.

[48] T. Kwartler, "The OpenNLP project," in *Text Mining in Practice With R*. Hoboken, NJ, USA: Wiley, 2017, pp. 237–269.

[49] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," 2017, *arXiv:1709.00103*.

[50] B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, R. Geng, R. Cao, J. Sun, L. Si, F. Huang, and Y. Li, "A survey on text-to-SQL parsing: Concepts, methods, and future directions," 2022, *arXiv:2208.13629*.

[51] G. Katsogiannis-Meimarakis and G. Koutrika, "A survey on deep learning approaches for text-to-SQL," *VLDB J.*, vol. 32, no. 4, pp. 905–936, Jul. 2023.

[52] J. Elman, "Finding structure in time," *Cognit. Sci.*, vol. 14, no. 2, pp. 179–211, Jun. 1990.

[53] S. Hochreiter, "Long short-term memory," in *Neural Computation*. Cambridge, MA, USA: MIT Press, 1997.

[54] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.

[55] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[56] A. Radford, "Improving language understanding by generative pre-training," 2018.

[57] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.

[58] J. Guo, Z. Zhan, Y. Gao, Y. Xiao, J.-G. Lou, T. Liu, and D. Zhang, "Towards complex text-to-SQL in cross-domain database with intermediate representation," 2019, *arXiv:1905.08205*.

[59] T. Yu, M. Yasunaga, K. Yang, R. Zhang, D. Wang, Z. Li, and D. Radev, "SyntaxSQLNet: Syntax tree networks for complex and cross-domainText-to-SQL task," 2018, *arXiv:1810.05237*.

[60] W. Hwang, J. Yim, S. Park, and M. Seo, "A comprehensive exploration on WikiSQL with table-aware word contextualization," 2019, *arXiv:1902.01069*.

[61] W. Lei, W. Wang, Z. Ma, T. Gan, W. Lu, M.-Y. Kan, and T.-S. Chua, "Re-examining the role of schema linking in text-to-SQL," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6943–6954.

[62] J. Yi and G. Chen, "Decoupling SQL query hardness parsing for text-to-SQL," 2023, *arXiv:2312.06172*.

[63] N. Yaghmazadeh, Y. Wang, I. Dillig, and T. Dillig, "SQLizer: Query synthesis from natural language," in *Proc. ACM Program. Lang.*, vol. 1, 2017, pp. 1–26.

[64] B. Bogin, M. Gardner, and J. Berant, "Representing schema structure with graph neural networks for text-to-SQL parsing," 2019, *arXiv:1905.06241*.

[65] X. Xu, C. Liu, and D. Song, "SQLNet: Generating structured queries from natural language without reinforcement learning," 2017, *arXiv:1711.04436*.

[66] M. Liu, B. Liang, M. Zhu, and C. Chu, "Stochastic check-in employee scheduling problem," *IEEE Access*, vol. 8, pp. 80305–80317, 2020, doi: 10.1109/ACCESS.2020.2990340.

[67] O. Rubin and J. Berant, "SmBoP: Semi-autoregressive bottom-up semantic parsing," 2020, *arXiv:2010.12412*.

[68] Y. Deng, Z. Lin, Y. Zou, J. Tan, X. Huang, and D. Ji, "MetaSQL: Improving text-to-SQL semantic parsing with meta-learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, pp. 1–12.

[69] Y. Zhang, Z. Deng, Y. Zou, J. Tan, X. Huang, and D. Ji, "E-SQL: Enhancing text-to-SQL semantic parsing with entity linking," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, pp. 1021–1030.

[70] Z. Yao, Y. Su, H. Sun, and W.-T. Yih, "Model-based interactive semantic parsing: A unified framework and a text-to-SQL case study," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 5447–5458.

[71] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[72] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[73] OpenAI. (2022). *ChatGPT: Optimizing Language Models for Dialogue*. OpenAI Blog. [Online]. Available: https://openai.com/blog/chatgpt/

[74] S. Dhuliawala, M. Sachan, and C. Allen, "Variational classification," 2023, *arXiv:2305.10406*.

[75] R. Rahman, R. Hasan, A. A. Farhad, M. T. R. Laskar, M. H. Ashmafee, and A. R. M. Kamal, "ChartSumm: A comprehensive benchmark for automatic chart summarization of long and short summaries," 2023, *arXiv:2304.13620*.

[76] N. Ho, L. Schmid, and S.-Y. Yun, "Large language models are reasoning teachers," 2022, *arXiv:2212.10071*.

[77] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[78] T. Yu, R. Zhang, M. Yasunaga, Y. C. Tan, X. V. Lin, S. Li, H. Er, I. Li, B. Pang, T. Chen, E. Ji, S. Dixit, D. Proctor, S. Shim, J. Kraft, V. Zhang, C. Xiong, R. Socher, and D. Radev, "SParC: Cross-domain semantic parsing in context," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, Florence, Italy, 2019, pp. 4511–4523.

[79] B. Guo, Z. Zhang, and X. Liu, "Sequence-to-SQL generation by self-attention and pointer networks," *IEEE Access*, vol. 8, pp. 34878–34888, 2020.

[80] D. Gao and B. Li, "Text-to-SQL empowered by large language models: A benchmark evaluation," 2023, *arXiv:2305.03111*.

[81] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023, pp. 1–10.

[82] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL+GNN: Enhancing text-to-SQL models with graph neural networks," 2020, *arXiv:2004.03231*.

[83] R. Sun, S. Goyal, J. Gu, J. Liu, M. Bendersky, M. Najork, and Y. Zhang, "SQL-PaLM: Improved large language model adaptation for text-to-SQL," 2023, *arXiv:2306.00739*.

HASSAN S. SHALAAN received the bachelor's and M.Sc. degrees in information systems, in 2010 and 2018, respectively. He was appointed as a Teaching Assistant, in 2012. He is currently an Assistant Lecturer with the Department of Information Systems, Faculty of Computers and Information, Assiut University, Egypt. He is also working on schema-aware approaches for converting natural language queries into executable SQL statements. He is a proficient in database technologies and NLP tools, such as Stanford CoreNLP and GloVe, with practical experience in Python, SQL, and deep learning frameworks. His academic and research interests include natural language interfaces to databases (NLIDB), natural language processing (NLP), hybrid rule-based and deep learning models, and intelligent query translation.

TAYSEER HASSAN A. SOLIMAN (Member, IEEE) was born in Assiut, Egypt. She received the B.Sc. degree (Hons.) in computer science from Marymount University, USA, in 1990, the M.Sc. degree in computational sciences from the Faculty of Science, Cairo University, in 1996, and the joint Ph.D. degree in bioinformatics from the Faculty of Computers and Information Science, Ain Shams University, Egypt, and the University of Connecticut, USA, in 2004. She was an Assistant Professor with the Faculty of Computers and Information, Ain Shams University, from 2004 to 2006. She was a Visiting Scientist with IBM, CAS Center, Cairo Division, from May 2005 to 2007. She is currently the Dean of Faculty of Computers and Information, Assiut University, Egypt. She is also the supervisor of data science, scientific data management, bioinformatics, big data management and analytics, and language technologies research groups and laboratories with the faculty. She supervised and currently supervising a number of national and international research projects as Assiut University PI under Erasmus+, DAAD, and STDF, Egypt. She published more than 65 papers in national/international conferences and journals and supervised 50 M.Sc. and Ph.D. theses. Her research interests include data mining, data science, semantic data integration, biodiversity informatics, bioinformatics, language technologies, big data management, and analytics. She is a member of IEEE WIE, ACM SIGKDD, ACM SIGBIO, and Unesco ICT Committee. She served community and strategic levels, such as Egypt AI Initial Strategic Plan Main Committee.

AMR M. ABDELAZIZ received the Ph.D. degree in intelligent multi-objective data mining for big data applications. He is currently an Assistant Professor with the Information Systems Department, Beni-Suef University, Egypt. He has held several key leadership roles, including the Chief Information Officer with Beni-Suef University, where he spearheaded IT infrastructure development and digital transformation initiatives. He has supervised numerous postgraduate research projects in areas, such as document summarization, fake news detection, and natural language inference, leveraging advanced deep learning models, such as BERT and AraBERT. He has also contributed to nationally funded research projects on COVID-19 crisis management, supported by KACST and STDF. His scholarly work has been published in several high-impact, peer-reviewed journals, including *Journal of King Saud University-Computer and Information Sciences*, *Algorithms* (MDPI), and *PeerJ Computer Science*. His research interests include natural language processing, image analysis, big data analytics, and optimization algorithms.